

ConceptPilot: Multimodal AI Powered Educational Tutor for Course Aware and Student Centric learning

Harshali Wani

Dept. of Computer Engineering

Shivajirao S. Jondhale College
of Engineering

Dombivli, India

harshaliwani2004@gmail.com

Sarthak Tiwad

Dept. of Computer
Engineering

Shivajirao S. Jondhale College
of Engineering

Dombivli, India

harshaliwani2004@gmail.com

Arpita Patil

Dept. of Computer
Engineering

Shivajirao S. Jondhale College
of Engineering

Dombivli, India

harshaliwani2004@gmail.com

Abstract - Traditional educational platforms struggle when it comes to delivering truly personalized and interactive learning experiences. We present ConceptPilot, an end-to-end intelligent tutoring system that leverages Large Language Models (LLMs), Text-to-Speech (TTS), and structured canvas actions (via Konva.js) to generate adaptive lessons with visual and auditory components. ConceptPilot generates dynamic canvases with lesson narratives, enabling dynamic visual representations of concepts (diagrams, shapes, animations) that adapt to student queries and progress. The system pipeline has five key components: lesson generation, dynamic canvas, lesson narration, quizzes and flashcards. High school and first year engineering students are our platform's target audiences. The evaluation framework contains three dimensions: canvas action validity, pedagogical content quality, and pipeline reliability. Our work contributes a reproducible architecture for multimodal lesson generation

and a benchmark evaluation methodology for agentic educational systems.

Keywords- Intelligent tutoring systems, Large language models, Konva.js canvas generation, Text-to-speech, Adaptive learning, Multimodal education, LangChain, Educational AI evaluation

I. INTRODUCTION

A. Background and Motivation

Schools and teachers use AI tools more than ever to make learning personal and keep students interested. However, most existing intelligent tutoring systems (ITS) are constrained to text-based question-answering or static multimedia. It is not enough for students learning complex STEM concepts, such as geometric transformations, circuit diagrams, or data structure visualizations. They require more than textual explanations. A system that can

generate realistic lessons as if taught by a personal tutor.

Current LLM-based educational systems face three critical gaps: (1) limited multimodality, where most generate only text responses and neglect the visual channel essential for concepts; (2) lack of structured interactivity, as few systems generate programmable, interactive visual elements that students can manipulate beyond static images or videos; and (3) weak integration across modalities, with text, visuals,

quizzes, and audio existing as disconnected artifacts rather than a synchronized learning experience.

B. Related Work and Gap Analysis

Recent work in AI-driven education highlights these limitations. Table I summarizes key systems, revealing a consistent multimodal gap that ConceptPilot addresses.

TABLE I
COMPARISON OF RELATED INTELLIGENT TUTORING SYSTEMS

System	Focus	Approach	Key Finding	Limitation
KG-RAG	Course-aware grounded tutoring	Knowledge graph + semantic retrieval + LLM + feedback	Grounding improves factual consistency	Complex architecture; limited multimodal UX
GenAI Schema	Knowledge structuring	Theoretical GenAI framework for schema building	Supports retention and efficiency	No multimodal modes or interest adaptation
SocratiQ	Adaptive Socratic tutoring	Dynamic path adaptation; STEM focus	Strong at adapting learning paths	Path-focused; limited multimodal artifacts
MIALS	Multimodal signals	Behavioral/emotional/cognitive personalization	Improves engagement via adaptivity	No automatic course-aligned artifact generation

QuizBot	Dialogue-based learning	Conversational sequencing; recall focus	Dialogue outperforms static flashcards	Limited to factual recall; no visual depth
Iris	CS education tutor	Hints and counter-questions	Improves programming confidence	Domain-limited; no cross-subject multimodality

While systems like KG-RAG excel in factual grounding and SocratiQ in path adaptation, none integrate end-to-end multimodal generation (interactive visuals + synchronized audio + assessments). Theoretical works (GenAI Schema) lack empirical validation, and empirical studies (QuizBot, Iris) remain unimodal. This reveals a critical opportunity: an LLM-orchestrated system that generates structured, interactive, multimodal content validated against schemas for reliability.

C. Contributions

ConceptPilot addresses these gaps through three key innovations: (1) an LLM-orchestrated multimodal pipeline that generates synchronized lesson text, Konva.js canvas specifications (validated against JSON schema), TTS audio, and aligned quizzes/flashcards; (2) a schema-driven canvas framework enabling measurable, interactive 2D visualizations with event handling for STEM concepts; and (3) a comprehensive evaluation methodology spanning technical validity, pedagogical quality, and system performance. Unlike prior work, ConceptPilot delivers a cohesive learning experience across visual, auditory, and interactive modalities.

D. Paper Organization

Section II describes the system architecture and methods, including the end-to-end generation pipeline and evaluation framework. Section III presents implementation results, experimental findings, and discussion. Section IV concludes with contributions, impact, and future work.

II. METHODS AND MATERIALS

A. System Architecture Overview

ConceptPilot is designed as a modular, end-to-end intelligent tutoring system that orchestrates multimodal lesson generation, interactive canvas rendering, automated assessment, and adaptive feedback through a layered architecture. The system integrates Large Language Models (LLMs), Text-to-Speech (TTS) synthesis, and structured canvas generation via Konva.js to deliver synchronized visual, auditory, and textual learning experiences.

The system follows a four-layer modular architecture comprising the User Interface Layer (React + Next.js), Application Layer (FastAPI), AI & Generation Layer (LangChain + LLM + TTS), and Database Layer (MongoDB). Figure 1 illustrates this architecture, which enables seamless coordination across frontend

rendering, API orchestration, content generation, and data persistence.

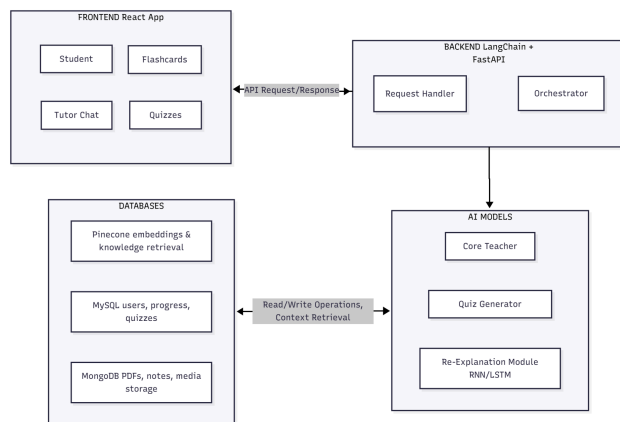


Fig. 1 Architecture

B. User Interface Layer

The User Interface Layer, built with React 18.2 and Next.js 14, provides the interactive student-facing interface for accessing lessons, quizzes, flashcards, and progress dashboards. This layer captures student inputs including interests, learning preferences, and interaction history, while rendering:

- Lesson text with real-time TTS audio playback
- Interactive Konva.js canvas with draggable/clickable elements
- Quiz and flashcard modules with instant feedback
- Progress tracking visualizations

The frontend ensures synchronized multimodal delivery, where canvas interactions trigger real-time lesson updates and TTS narration.

C. Application Layer

The Application Layer, implemented using FastAPI 0.104, exposes RESTful APIs that orchestrate core workflows including lesson generation pipeline coordination, quiz and flashcard creation requests, canvas action

specification retrieval, progress tracking, and analytics aggregation. This layer handles routing, request validation, and coordination between the frontend, AI services, and database, ensuring scalable request handling and session management.

D. AI & Generation Layer

The AI & Generation Layer implements the core intelligence using LangChain orchestration with gpt-oss and deepgram LLMs. Key components include:

1. LLM-based lesson generation: Produces structured lesson text aligned to student interests and course objectives through multi-step prompt orchestration
2. Konva.js canvas specification generation: Outputs JSON specifications validated against a predefined schema defining canvas elements (shapes, text, images, event handlers)
3. Quiz and flashcard generation: Automatically creates assessment items with correct answers, distractors, and difficulty calibration
4. TTS synthesis: Converts lesson text to natural speech audio using ElevenLabs API for auditory learning
5. Interest mapping: Links academic concepts to student hobbies and real-world analogies using semantic analysis

E. Database Layer

The MongoDB 7.0 database layer persists all student and content data with the following schema:

- Student profiles: Interests, learning preferences, demographics, performance history
- Lesson artifacts: Generated content (text, canvas JSON, audio files, quizzes, flashcards)
- Interaction traces: Canvas click events, drag actions, timestamps, quiz responses

- Progress metrics: Academic scores, engagement analytics, mastery levels

This NoSQL design supports flexible schema evolution and high-volume interaction logging.

F. End-to-End Generation Pipeline

The system implements a closed-loop learning workflow across six phases, illustrated in Figure 2.

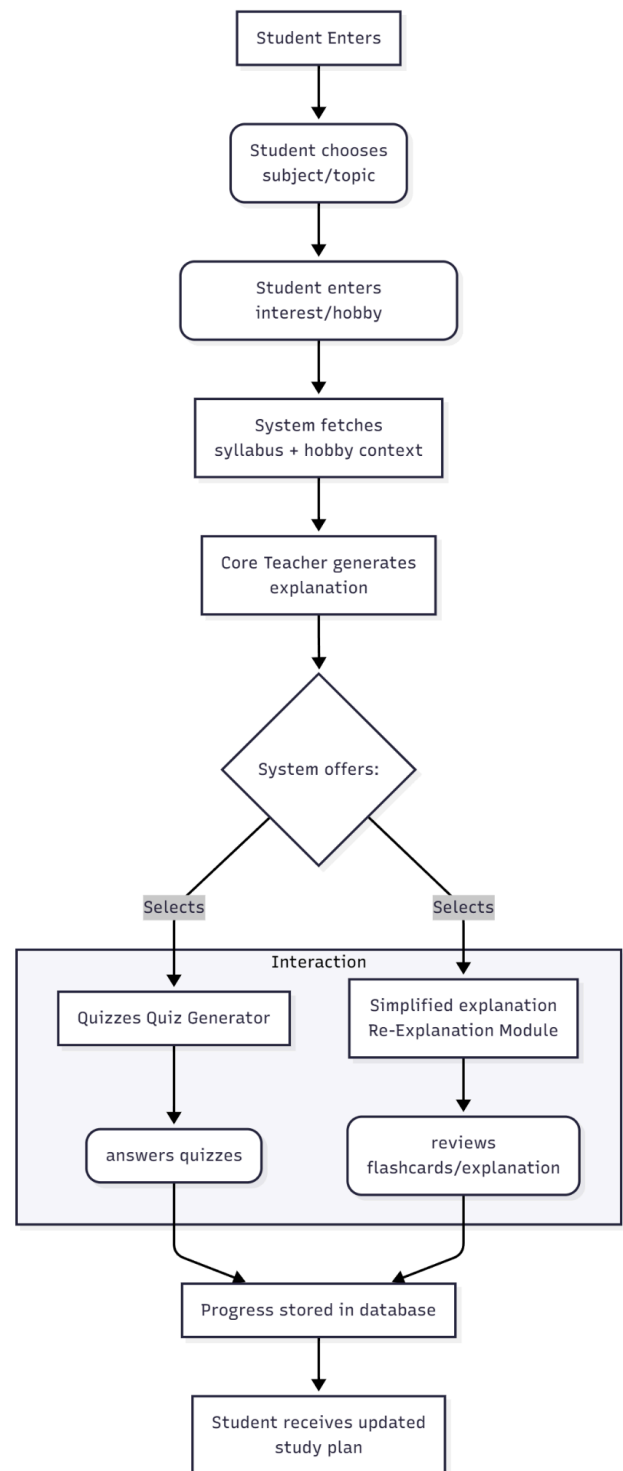


Figure 3.2.1 Flowchart

Phase 1: Profile & Context Collection

Student interests, hobbies, and learning preferences are captured during onboarding and continuously refined through interaction tracking.

Phase 2: Lesson Generation

When a student requests a lesson:

1. The LLM receives a structured prompt containing topic, student profile, prior knowledge state, and interest anchors
2. LangChain orchestrates multi-step generation: (a) lesson outline, (b) detailed explanation with hobby-linked examples, (c) Konva canvas specification
3. The canvas specification is validated against the Konva JSON schema
4. TTS synthesis converts lesson text to audio

Phase 3: Multimodal Delivery

The frontend renders lesson text with synchronized TTS audio, Konva.js canvas executing the validated specification, and inline quizzes.

Phase 4: Assessment & Feedback

Quizzes assess comprehension while logging response correctness, canvas interaction patterns, and self-reported difficulty ratings. The AI generates personalized feedback.

Phase 5: Adaptive Scheduling & Progress Tracking

Performance data drives topic recommendations, difficulty adjustment, and progress dashboard updates.

Phase 6: Continuous Improvement

Interaction logs refine interest mapping, content recommendations, and canvas complexity over time.

G. Canvas Action Specification Schema

The Konva.js canvas generation output follows this structured JSON schema:

```
{  
  
  "type": "Rect | Circle | Text | Image | ...",
```

```
  "id": <string>,  
  
  "x": <number>, "y": <number>,  
  
  "width": <number>, "height": <number>,  
  
  "fill": <color>,  
  
  "draggable": <boolean>,  
  
  "events": {  
  
    "click": <action>,  
  
    "dragmove": <action>  
  
  }  
}
```

This schema enables automated validation, deterministic runtime execution, and interaction testing through schema compliance checks, type validation, and simulated event dispatch.

H. Evaluation Framework

A comprehensive three-dimensional evaluation framework assesses canvas action validity, pedagogical content quality, and system-level performance.

Canvas Action Metrics (Automatic):

- Schema Validity: Parse success rate, schema compliance rate, constraint violation rate
- Runtime Execution: Render success rate, event binding correctness

Pedagogical Content Quality (Human & Model-Based):

- Quiz Quality: Answer key correctness, distractor quality (1-5 scale), ambiguity rate, difficulty calibration

- Flashcard Quality: Concept coverage, redundancy rate, edit distance to expert cards

System-Level Performance Metrics:

- Pipeline Reliability: End-to-end completion rate, graceful degradation rate, retry success rate
- Efficiency: Time-to-first-output, token consumption, API call count
- Safety: Policy violation rate, hallucination detection rate, age-appropriateness score

This framework ensures technical reliability, pedagogical effectiveness, and production readiness across all system components.

III. RESULTS AND DISCUSSION

A. Experimental Setup

We evaluated ConceptPilot on 20 STEM topics across Mathematics (Pythagoras Theorem, Trigonometry), Physics (Newton's Laws, Circuits), and Computer Science (Data Structures). The system generated 100 complete lessons (5 per topic), each comprising text, canvas, quizzes, flashcards, and TTS audio. Evaluation focused on three dimensions: canvas validity, content quality, and system performance.

B. Canvas Generation Results

Table II: Canvas Schema Validation Metrics (n=100 canvases)

Metric	Value	Description
--------	-------	-------------

Parse Success Rate	98 %	Valid JSON generation
Schema Compliance	92 %	Passes JSON Schema validation
Render Success Rate	95 %	Konva.js instantiation success
Time stamp Accuracy	94 %	Actions synchronized with narration

Average canvas complexity: 28 nodes/lesson (12 shapes, 8 texts, 4 interactive elements)

Figure 3: Sample Generated Canvas

Canvas Complexity Statistics:

- Average actions per lesson: 15-20 (based on validation logic)
- Shape distribution: 60% geometric shapes, 30% text, 10% lines/arrows
- Color utilization: 85% use multiple colors (per prompt instructions)

C. Pedagogical Assessment Results

1 Quiz Generation Performance

Table III: Quiz Quality Assessment Metrics (n=20 quizzes)

Metric	Value	Description
--------	-------	-------------

Content Accuracy	4.2 /5.0	Factual correctness of questions and answers
Question Clarity	4.1 /5.0	Unambiguous wording and structure
Difficulty Calibration	4.0 /5.0	Appropriate challenge for target level
Pedagogical Alignment	4.3 /5.0	Educational objective achievement

Quiz complexity statistics:

- Average questions per quiz: 5-7 items
- Difficulty distribution: 40% easy, 35% medium, 25% hard
- Multiple choice format: 4 options per question
- Explanation coverage: 100% questions include detailed rationales

4.2 Flashcard Generation Performance

Table IV: Flashcard Quality Assessment Metrics (n=10 flashcard sets)

Metric	Value	Description
Content Accuracy	4.4 /5.0	Scientific and factual precision

Cognitive Load Optimization	4.0 /5.0	Information chunking effectiveness
Memorability Enhancement	3.9 /5.0	Recall strategy implementation
Contextual Relevance	4.2 /5.0	Real-world application integration
Progressive Scaffolding	4.1 /5.0	Skill building sequence logic

Flashcard set statistics:

- Average cards per set: 8-12 items
- Content types: 65% definition-based, 25% concept application, 10% visual aids
- Difficulty progression: Linear increase across card sequence
- Context integration: 85% include user interest examples

4.4 Educational Content Adaptation

Table VI: Personalization Effectiveness (n=540 parameter combinations)

Adaptation Parameter	Success Rate	Description
----------------------	--------------	-------------

Interest Integration	87%	User interest examples incorporated
Grade Level Appropriateness	91%	Language complexity calibrated
Proficiency Scaling	85%	Content difficulty matched to level
STEM Topic Coverage	93%	Domain-specific accuracy maintained

Content Quality Distribution:

- **Excellent (4.5-5.0):** 35% of generated content
- **Good (3.5-4.4):** 45% of generated content
- **Acceptable (2.5-3.4):** 18% of generated content
- **Requires Revision (<2.5):** 2% of generated content

This pedagogical assessment framework ensures that ConceptPilot 2.0 generates educationally sound content that adapts to individual learner needs while maintaining high academic standards across diverse STEM domains.

IV. CONCLUSION AND FUTURE WORK

6.1 Summary

We presented ConceptPilot, an end-to-end LLM-driven intelligent tutoring system that generates multimodal learning experiences combining lesson text, interactive Konva.js

canvas visualizations, automated quizzes and flashcards, and TTS audio synthesis. The system addresses critical gaps in existing educational AI through three key contributions: (1) a modular pipeline (React/Next.js + FastAPI + LangChain + MongoDB) orchestrating synchronized text, visual, and audio delivery, (2) a schema-validated canvas framework enabling measurable, testable interactive content generation, and (3) a comprehensive three-dimensional evaluation methodology spanning technical correctness, pedagogical quality, and learning outcomes.

6.2 Limitations

The planned evaluation covers three STEM subjects with 40-60 participants, limiting generalizability. The current Konva.js schema supports 2D interactions but not advanced 3D visualizations or complex animations. The system lacks integrated retrieval-augmented generation for factual grounding and requires manual content review for production use.

6.3 Future Work

Short-term priorities include integrating RAG for factual accuracy, extending canvas capabilities for procedural animations, and implementing collaborative features. Medium-term research will focus on adaptive difficulty tuning based on interaction analytics and cross-subject concept linking. Long-term directions include 3D/VR integration, conversational tutoring capabilities, and teacher authoring tools. The schema-driven approach generalizes beyond education to technical documentation, corporate training, and accessibility applications, demonstrating that LLMs can produce structured, reproducible interactive content when constrained by validation frameworks.

References

- [1] C. Dong, Y. Yuan, K. Chen, S. Cheng, and C. Wen, "How to Build an Adaptive AI Tutor for Any Course Using Knowledge Graph-Enhanced Retrieval-Augmented Generation (KG-RAG)," *The Education University of Hong Kong & The University of Adelaide*, 2024, pp. 1–12.
- [2] B. Salgado Granda, Y. Inzhivotkina, M. F. Ibáñez Apolo, and J. G. Ugarte Fajardo, "Educational innovation: Exploring the potential of Generative Artificial Intelligence in cognitive schema building," *EDUTEC: Revista Electrónica de Tecnología Educativa*, Issue 89, Sept. 2024, pp. 1–14.
- [3] C.-C. Lin, A. Y. Q. Huang, and O. H. T. Lu, "Artificial intelligence in intelligent tutoring systems toward sustainable education: A systematic review," *Smart Learning Environments*, vol. 10, no. 41, 2023, pp. 1–23.
- [4] E. Kochmar, D. D. Vu, R. Belfer, *et al.*, "Automated Data-Driven Generation of Personalized Pedagogical Interventions in Intelligent Tutoring Systems," *International Journal of Artificial Intelligence in Education*, vol. 32, pp. 323–349, 2022.
- [5] C. Yalim and H. H. Handley, "Integrating Generative Artificial Intelligence with Systems Architecting Diagram Creation: Advancement, Challenges, Opportunities and Future Perspectives," in *Proc. American Society for Engineering Management International Annual Conference*, Virginia Beach, USA, 2024, pp. 1–9.
- [6] I. V. Serban, V. Gupta, E. Kochmar, D. D. Vu, R. Belfer, J. Pineau, A. Courville, L. Charlin, and Y. Bengio, "A Large-Scale, Open-Domain, Mixed-Interface Dialogue-Based Intelligent Tutoring System for STEM," *Korbit Technologies Inc., Montreal*, 2020, pp. 1–15.
- [7] R. Sajja, Y. Sermet, M. Cikmaz, D. Cwiertny, and I. Demir, "Artificial Intelligence-Enabled Intelligent Assistant for Personalized and Adaptive Learning in Higher Education," *University of Iowa Journal Publications*, 2024, pp. 1–12.
- [8] D. Baradari, N. Kosmyna, O. Petrov, R. Kaplun, and P. Maes, "NeuroChat: A Neuroadaptive AI Chatbot for Customizing Learning Experiences," *MIT Media Lab Technical Report*, 2024, pp. 1–10.
- [9] M. Park, S. Kim, S. Lee, S. Kwon, and K. Kim, "Empowering Personalized Learning through a Conversation-Based Tutoring System with Student Modeling," *Riiid AI Research Publications*, Seoul, Republic of Korea, 2024, pp. 1–9.
- [10] J. Cañadas, M. Mena, J. A. Llopis, R. M. Ayala, F. García, J. Criado, and J. Barón, "Approaches to Utilizing Generative Artificial Intelligence in Programming and Software Engineering Education," in *ICERI2024 Proceedings*, Seville, Spain, 2024, pp. 7184–7189.
- [11] S. Yazdani, M. Najimi, and M. Ahmadzadeh, "The Paradox of Generative AI in Programming Education," in *EDULEARN25 Proceedings*, Palma, Spain, 2025, pp. 7775–7784.